CIN0136 — Desenvolvimento de Software

EXERCÍCIO ESCOLAR 1 — ESPELHO DE CORREÇÃO

Material de revisão e aprendizado | 2026.2

Semanas 1–7
13/10/2026
Profs. KIEV & VCG

Como usar este material

Este documento é o espelho do Exercício Escolar 1. Para cada questão você encontrará: o enunciado original, a resposta modelo completa e uma explicação pedagógica que conecta o tema à prática do projeto. Use-o para:

- Comparar sua resposta com o esperado, identifique o que acertou, o que ficou incompleto e o que errou.
- Ler as explicações pedagógicas, elas revelam o raciocínio por trás da resposta correta, não apenas o resultado.
- Identificar erros comuns, para não repeti-los nos próximos sprints e no Exercício Escolar 2.
- Conectar o conteúdo teórico ao projeto real, cada gabarito menciona como o conceito aparece no seu trabalho de extensão.

Regras de correção (modelo somatório com nota parcial)

- Cada questão apresenta de 5 a 7 alternativas, rotuladas por potência de 2 a partir de **2** (2, 4, 8, 16, 32, 64, 128) conforme a **posição**. O aluno soma os valores das alternativas que julga **verdadeiras**.
- **Cada questão pontuada vale 1,0. Total: 10,0.** A **Q11 é bônus** e soma por cima (a nota pode ultrapassar 10,0).
- **Marcou qualquer alternativa FALSA → a questão vale 0,0**, independentemente de quantas verdadeiras também tenha marcado.
- Marcou **só verdadeiras, mas não todas** → nota proporcional: (nº de corretas marcadas ÷ nº total de corretas da versão) × 1,0.
- Marcou **todas as corretas e nenhuma falsa** → 1,0.
- **Arredondar apenas a nota final da prova** (2 casas decimais). Não arredondar por questão.
- O **nº total de corretas pode variar entre versões** de uma mesma questão — use sempre o denominador da versão aplicada (ver Conferência final de gabaritos).

QUESTÃO 1 — Fundamentos de ES: programar vs. desenvolver software

(5 alternativas · 2 corretas em A/B/C, 3 em D · 1,0 ponto)

Cenário (A/B/C): A equipe que atende a ONG **Raízes Urbanas** recebeu, de um voluntário estudante de programação, um script em Python que gera os relatórios mensais de doações. O script funciona na máquina do voluntário e produz o relatório correto em segundos. A coordenadora pediu para incorporá-lo ao sistema (que usa Node.js). Ao abri-lo, a equipe encontra um único arquivo de 300 linhas, sem testes, sem comentários, com regras de cálculo e acesso ao banco misturados na mesma função. O voluntário se formou e não está mais disponível. *Ruídos:* linguagem diferente do restante do sistema; o voluntário levou "duas tardes".

Versão A — Gabarito: 12 (4 + 8)

Pos	Val	V/F	Alternativa (resumo)	Justificativa
1ª	2	**F**	Python (≠ Node) é a principal razão de não ser artefato de qualidade	Eleva um problema secundário (heterogeneidade de linguagem) a causa principal. O cerne da falta de qualidade é testabilidade/manutenibilidade ausentes, que existiriam em qualquer linguagem.
2ª	4	**V**	"Funcionar" é insuficiente; faltam manutenibilidade e testabilidade p/ evolução	Conecta *funcionar ≠ qualidade* à consequência concreta (evolução futura depende desses atributos).
3ª	8	**V**	Dificuldade de modificar: desafio técnico **e** organizacional	Conecta as duas dimensões dos desafios de ES, ancoradas no cenário (código sem estrutura + conhecimento que saiu com a pessoa).
4ª	16	**F**	Incorporar já; reorganizar seria "otimização prematura"	Invoca princípios reais (entregar valor cedo; evitar otimização prematura) para justificar decisão errada. "Otimização prematura" é sobre desempenho, não sobre estruturar código ilegível/não testável.
5ª	32	**F**	Conferência manual da saída dispensa testes	Separa "cobertura de testes" do "risco de regressão", quando é esse risco que os testes endereçam. Verificação pontual ≠ garantia sob mudança.

Versão B — Gabarito: 18 (2 + 16)

Pos	Val	V/F	Conteúdo (= alternativa de A)
1ª	2	**V**	[A-3ª] Desafio técnico + organizacional
2ª	4	**F**	[A-1ª] Python é a principal razão
3ª	8	**F**	[A-5ª] Conferência dispensa testes
4ª	16	**V**	[A-2ª] "Funcionar" é insuficiente
5ª	32	**F**	[A-4ª] Incorporar já; "otimização prematura"

Reordenação da A; justificativas idênticas às da A por alternativa.

Versão C — Gabarito: 36 (4 + 32)

Alteração: a falsa A-1ª (Python) foi substituída por uma **falsa mais sutil** sobre coesão/legibilidade.

Pos	Val	V/F	Conteúdo	Justificativa (só da nova)
1ª	2	**F**	[A-4ª] Incorporar já; "otimização prematura"	—
2ª	4	**V**	[A-2ª] "Funcionar" é insuficiente	—
3ª	8	**F**	**NOVA:** concentrar cálculo + acesso ao banco numa função de 300 linhas *favorece a legibilidade* por manter tudo num só lugar	Inverte a relação coesão/legibilidade. Misturar responsabilidades (cálculo + persistência) é o oposto de boa organização; "ver tudo junto é mais fácil de ler" é intuição de iniciante.
4ª	16	**F**	[A-5ª] Conferência dispensa testes	—
5ª	32	**V**	[A-3ª] Desafio técnico + organizacional	—

Versão D — Gabarito: 42 (2 + 8 + 32) · 3 corretas

Alteração de cenário: o relatório passa a ser gerado **sob demanda, dentro do próprio servidor Node.js em produção, a cada requisição** (execução in-process). Isso torna a incompatibilidade de *runtime* um obstáculo real → a alternativa sobre linguagem muda **F→V**.

Pos	Val	V/F	Conteúdo	Justificativa da que mudou
1ª	2	**V**	[A-3ª] Desafio técnico + organizacional	—
2ª	4	**F**	[A-5ª] Conferência dispensa testes	—
3ª	8	**V**	[A-2ª] "Funcionar" é insuficiente	—
4ª	16	**F**	[A-4ª] Incorporar já; "otimização prematura"	—
5ª	32	**V**	Rodar Python in-process no runtime Node a cada requisição é obstáculo	**Mudou F→V:** com execução in-process, a incompatibilidade de runtime deixa de ser secundária e

Pos	Val	V/F	Conteúdo	Justificativa da que mudou
			técnico real de integração (runtimes não compartilham ambiente)	vira barreira concreta. Afirma incompatibilidade de *runtime/processo* (verdadeira), não "linguagem é a principal razão da falta de qualidade".

QUESTÃO 2 — Modelos de processo: cascata, incremental e iterativo

(6 alternativas · 3 corretas em A/B/C, 2 em D · 1,0 ponto)

Cenário (A/B/C): Caso de aula. App de cadastro de beneficiários de uma ONG. **Abordagem 1:** entrega semana a semana partes novas e completas (nome; depois endereço; depois histórico) — cresce por adição. **Abordagem 2:** entrega o sistema inteiro simples na 1ª semana e vai refinando (validação, interface, feedback) — cresce por refinamento. *Ruídos:* integrante sem experiência com BD; coordenadora só dá feedback a partir da 3ª semana.

Versão A — Gabarito: 38 (2 + 4 + 32)

Pos	Val	V/F	Alternativa (resumo)	Justificativa
1ª	2	**V**	Abord. 1 = incremental (adição); Abord. 2 = iterativo (refinamento)	Distinção correta incremental × iterativo, ancorada nos dois casos.
2ª	4	**V**	Ambas entregam ao longo do tempo, ≠ cascata (validação tardia)	Conecta os dois modelos ao contraste com cascata.
3ª	8	**F**	Abord. 2 é preferível em **qualquer** projeto (refinar é sempre menos arriscado)	Generalização indevida; nenhum dos modelos é universalmente superior.
4ª	16	**F**	Abord. 2 = cascata por entregar "sistema inteiro" cedo	Confunde "sistema inteiro simples e refinável" (iterativo) com "sistema completo de uma vez ao final" (cascata). Cascata não entrega cedo nem refina.
5ª	32	**V**	Iterativo: cada ciclo assume aprendizado; feedback é entrada planejada	Usa o ruído (feedback só na 3ª semana) corretamente: feedback é planejado, ainda que atrasado.
6ª	64	**F**	Cascata é "erro histórico" sem contexto legítimo hoje	Caricatura que a Semana 2 desconstrói: cascata tem contextos

Pos	Val	V/F	Alternativa (resumo)	Justificativa
				legítimos (sistemas críticos, escopo fechado, alto custo de mudança).

Versão B — Gabarito: 84 (4 + 16 + 64)

Pos	Val	V/F	Conteúdo (= A)
1ª	2	**F**	[A-6ª] Cascata = erro histórico
2ª	4	**V**	[A-1ª] Abord.1 incremental / Abord.2 iterativo
3ª	8	**F**	[A-4ª] Abord.2 = cascata
4ª	16	**V**	[A-5ª] Iterativo: feedback como entrada
5ª	32	**F**	[A-3ª] Abord.2 preferível em qualquer projeto
6ª	64	**V**	[A-2ª] Ambas ≠ cascata (validação tardia)

Versão C — Gabarito: 82 (2 + 16 + 64)

Alteração: a falsa A-4ª foi substituída por **falsa mais sutil** que confunde incremental com iterativo via feedback.

Pos	Val	V/F	Conteúdo	Justificativa (só da nova)
1ª	2	**V**	[A-1ª] Abord.1 incremental / Abord.2 iterativo	—
2ª	4	**F**	[A-6ª] Cascata = erro histórico	—
3ª	8	**F**	**NOVA:** Abord. 1 incorpora feedback melhor, pois cada incremento é *revisto e retrabalhado sobre si mesmo*	"Revisto e retrabalhado sobre si mesmo" é refinamento (marca do **iterativo**, Abord. 2). O incremental adiciona partes *novas*; não pressupõe retrabalhar o incremento anterior.
4ª	16	**V**	[A-2ª] Ambas ≠ cascata	—
5ª	32	**F**	[A-3ª] Abord.2 preferível em qualquer projeto	—
6ª	64	**V**	[A-5ª] Iterativo: feedback como entrada	—

Versão D — Gabarito: 10 (2 + 8) · 2 corretas

Alteração de cenário: a **Abordagem 1** passa a entregar o sistema inteiro simples e a *refiná-lo* (vira iterativa). A alternativa que a classifica como incremental muda **V→F**.

Pos	Val	V/F	Conteúdo	Justificativa da que mudou
1ª	2	**V**	[A-5ª] Iterativo: feedback como entrada	—
2ª	4	**F**	[A-3ª] Abord.2 preferível em qualquer projeto	—
3ª	8	**V**	[A-2ª] Ambas ≠ cascata	—
4ª	16	**F**	[A-6ª] Cascata = erro histórico	—
5ª	32	**F**	Abord. 1 = incremental (adição de partes novas); Abord. 2 = iterativo	**Mudou V→F:** no cenário D, a Abord. 1 passou a refinar a mesma versão → virou iterativa. Classificá-la como incremental está agora incorreto; as duas são iterativas.
6ª	64	**F**	[A-4ª] Abord.2 = cascata	—

QUESTÃO 3 — Scrum e Kanban: eventos, papéis e fluxo

(6 alternativas · 4 corretas em A/B/C, 3 em D · 1,0 ponto)

Cenário (A/B/C): Equipe da ONG **Raízes Urbanas** encerrando a 3ª Sprint (Scrum, sprints de 2 semanas, board no GitHub Projects). Entregaram o cadastro de voluntários e corrigiram 3 bugs. O integrante que **guarda e prioriza o backlog** notou muitos itens simultâneos em "Em progresso". *Ruídos:* desentendimento sobre editor de código; coordenadora confirmou presença no fechamento.

Versão A — Gabarito: 90 (2 + 8 + 16 + 64)

Pos	Val	V/F	Alternativa (resumo)	Justificativa
1ª	2	**V**	Review = incremento com stakeholder; Retrospectiva = processo (interna)	Distingue os dois eventos de fechamento.
2ª	4	**F**	Usar o fechamento + presença da coordenadora p/ mediar o conflito de equipe	**Âncora.** Mistura Review (incremento, com stakeholder) com assunto de Retrospectiva (processo), que é interna

Pos	Val	V/F	Alternativa (resumo)	Justificativa
				e sem stakeholder. Usa o ruído (conflito + presença externa).
3ª	8	**V**	Muitos itens em "Em progresso" contraria limite de WIP; reduzir baixa o cycle time	Conecta WIP a cycle time.
4ª	16	**V**	Guardião/priorizador do backlog = Product Owner	Identifica o PO pela responsabilidade, não pelo nome.
5ª	32	**F**	Board Kanban e Sprints do Scrum são mutuamente exclusivos	Falsa exclusividade: board Kanban é usado como visualização *dentro* do Scrum (o próprio GitHub Projects).
6ª	64	**V**	Daily = sincronização (feito/farei/impedimentos) ; não substitui a Review	Distingue Daily de Review sem colapsá-los.

Versão B — Gabarito: 108 (4 + 8 + 32 + 64)

Pos	Val	V/F	Conteúdo (= A)
1ª	2	**F**	[A-5ª] Kanban e Scrum exclusivos
2ª	4	**V**	[A-1ª] Review vs. Retrospectiva
3ª	8	**V**	[A-6ª] Daily não substitui Review
4ª	16	**F**	[A-2ª] Fechamento p/ mediar conflito (âncora)
5ª	32	**V**	[A-3ª] WIP alto; reduzir baixa o cycle time
6ª	64	**V**	[A-4ª] Guardião do backlog = Product Owner

Versão C — Gabarito: 114 (2 + 16 + 32 + 64)

Alteração: a falsa A-5ª foi substituída por **falsa mais sutil** (WIP substitui o Planning).

Pos	Val	V/F	Conteúdo	Justificativa (só da nova)
1ª	2	**V**	[A-1ª] Review vs. Retrospectiva	—
2ª	4	**F**	[A-2ª] Fechamento p/ mediar conflito (âncora)	—

Pos	Val	V/F	Conteúdo	Justificativa (só da nova)
3ª	8	**F**	**NOVA:** aplicar limite de WIP torna o Sprint Planning *dispensável* (cumpriria o mesmo papel de definir o compromisso do ciclo)	Aceita a boa prática (WIP no Scrum) para errar: WIP controla *fluxo* (itens simultâneos); Planning define *compromisso de escopo* do ciclo. Propósitos distintos; um não substitui o outro.
4ª	16	**V**	[A-3ª] WIP alto; reduzir baixa o cycle time	—
5ª	32	**V**	[A-6ª] Daily não substitui Review	—
6ª	64	**V**	[A-4ª] Guardião do backlog = Product Owner	—

Versão D — Gabarito: 22 (2 + 4 + 16) · 3 corretas

Alteração de cenário: o integrante mencionado passa a **garantir as cerimônias e remover impedimentos** (papel de **Scrum Master**), não a guardar/priorizar o backlog. A alternativa que o chama de Product Owner muda **V→F**.

Pos	Val	V/F	Conteúdo	Justificativa da que mudou
1ª	2	**V**	[A-6ª] Daily não substitui Review	—
2ª	4	**V**	[A-3ª] WIP alto; reduzir baixa o cycle time	—
3ª	8	**F**	[A-2ª] Fechamento p/ mediar conflito (âncora)	—
4ª	16	**V**	[A-1ª] Review vs. Retrospectiva	—
5ª	32	**F**	Papel descrito (garantir cerimônias, remover impedimentos) = **Product Owner**	**Mudou V→F:** as responsabilidades descritas são de **Scrum Master**. Rotular como Product Owner está incorreto para o novo cenário.
6ª	64	**F**	[A-5ª] Kanban e Scrum exclusivos	—

QUESTÃO 4 — Requisitos funcionais, não funcionais e elicitação

(5 alternativas · 3 corretas em A/B/C, 4 em D · 1,0 ponto)

Cenário (A/B/C): Levantamento de requisitos do **PetFood**. Observando a atendente, a equipe vê que ela mantém um caderno com preferências de marca dos clientes — informação que o sistema atual não guarda. O dono diz que o sistema "precisa ser rápido" e que "qualquer funcionário novo tem que conseguir usar sem treinamento". Registrou-se também: cadastrar produtos com nome, preço e categoria. *Ruídos:* planos de abrir 2ª loja; caderno era presente de aniversário.

Versão A — Gabarito: 42 (2 + 8 + 32)

Pos	Val	V/F	Alternativa (resumo)	Justificativa
1ª	2	**V**	"Cadastrar produtos" (RF, verificável por presença) × "usar sem treinamento" (RNF cuja verificação exige definir o que conta como "sem treinamento")	Contrasta RF e RNF e liga o RNF à sua verificabilidade. Não é classificação isolada.
2ª	4	**F**	"Usar sem treinamento" é requisito **funcional** (envolve um usuário)	Usabilidade é RNF. Envolver usuário não torna o requisito funcional; RF = o que o sistema faz, RNF = como se comporta.
3ª	8	**V**	Observação expõe o tácito que a entrevista com o dono não alcançaria	Conecta duas técnicas (observação × entrevista) explicando por que uma revela o que a outra não.
4ª	16	**F**	"Rápido" já está pronto para teste objetivo, sem refinamento com o cliente	Sem critério mensurável (ex.: tempo-limite), não há o que verificar. RNFs vagos precisam ser refinados antes de virarem testáveis.
5ª	32	**V**	Caderno = requisito não atendido; surgiu da observação → limite de "o que você quer?"	Conecta o requisito concreto à limitação da elicitação por pergunta direta.

Versão B — Gabarito: 52 (4 + 16 + 32)

Pos	Val	V/F	Conteúdo (= A)
1ª	2	**F**	[A-4ª] "Rápido" pronto para teste

Pos	Val	V/F	Conteúdo (= A)
2ª	4	**V**	[A-1ª] RF × RNF e verificabilidade
3ª	8	**F**	[A-2ª] "Sem treinamento" é funcional
4ª	16	**V**	[A-5ª] Caderno / limite de "o que você quer?"
5ª	32	**V**	[A-3ª] Observação × entrevista

Versão C — Gabarito: 50 (2 + 16 + 32)

Alteração: a falsa A-2ª foi substituída por **falsa mais sutil** (RNF de usabilidade "necessariamente subjetivo").

Pos	Val	V/F	Conteúdo	Justificativa (só da nova)
1ª	2	**V**	[A-1ª] RF × RNF e verificabilidade	—
2ª	4	**F**	[A-4ª] "Rápido" pronto para teste	—
3ª	8	**F**	**NOVA:** por ser RNF de usabilidade, "usar sem treinamento" tem verificação *necessariamente subjetiva*, a cargo da impressão de cada funcionário; a equipe não deve traduzi-lo em critério observável	Acerta a classificação (RNF de usabilidade) para errar no encaminhamento: RNFs devem ser tornados verificáveis (ex.: completar a tarefa X em Y min sem ajuda). "Necessariamente subjetivo" contraria a medição de qualidade.
4ª	16	**V**	[A-3ª] Observação × entrevista	—
5ª	32	**V**	[A-5ª] Caderno / limite de "o que você quer?"	—

Versão D — Gabarito: 58 (2 + 8 + 16 + 32) · 4 corretas

Alteração de cenário: o dono especifica **"a busca tem que retornar em no máximo 2 segundos"**. O requisito de desempenho ganha critério mensurável → a alternativa que o tratava como não-verificável muda **F→V**.

Pos	Val	V/F	Conteúdo	Justificativa da que mudou
1ª	2	**V**	[A-5ª] Caderno / limite de "o que você quer?"	—
2ª	4	**F**	[A-2ª] "Sem treinamento" é funcional	—
3ª	8	**V**	[A-1ª] RF × RNF e verificabilidade	—
4ª	16	**V**	[A-3ª] Observação × entrevista	—
5ª	32	**V**	O requisito de desempenho, como expresso, traz critério mensurável e é verificável objetivamente, sem refinamento adicional	**Mudou F→V:** "rápido" virou "máximo 2 segundos" — critério mensurável. O RNF passou a ser testável.

QUESTÃO 5 — User Stories: INVEST, MoSCoW, Gherkin, hierarquia

(7 alternativas · 3 corretas · 1,0 ponto)

Cenário (A/B/C): Equipe da ONG **Raízes Urbanas** refinando o backlog. Itens: **(I)** "Como coordenadora, quero gerenciar voluntários, doações, eventos e relatórios para administrar a ONG"; **(II)** "Como voluntária, quero redefinir minha senha..."; **(III)** "Como desenvolvedor, quero migrar o banco para PostgreSQL para melhorar a performance". A coordenadora quer classificar "quase tudo" como prioridade máxima. *Ruídos:* reunião remarcada; discussão Trello vs. GitHub Projects.

Versão A — Gabarito: 26 (2 + 8 + 16)

Pos	Val	V/F	Alternativa (resumo)	Justificativa
1ª	2	**V**	Item (I) viola *Small*; é Epic; quebrar antes de estimar	Liga violação de *Small* à hierarquia (Epic × Story) e à consequência prática.
2ª	4	**F**	Item (III) é boa story porque segue o formato, e o formato *garante* valor	Segue o formato mas viola *Valuable* (tarefa técnica; persona "desenvolvedor" denuncia). Forma correta não implica valor.

Pos	Val	V/F	Alternativa (resumo)	Justificativa
3ª	8	**V**	Gherkin de caminho triste para (II) (link expirado → erro + novo link) é legítimo	Reconhece cenário de falha bem-formado e o valor do caminho triste.
4ª	16	**V**	Item (III) falha em *Valuable* (benefício técnico ≠ valor ao usuário) → é task, não story de produto	Identifica o critério INVEST correto e a consequência. Contrasta com a 2ª (que erra o mesmo diagnóstico).
5ª	32	**F**	MoSCoW: registrar quase tudo como *Must* (é a categoria do que o cliente acha importante)	"Se tudo é Must, nada é Must". *Must* = sem o qual o produto não funciona, não "tudo que o cliente valoriza".
6ª	64	**F**	*Won't Have (now)* = descartar definitivamente do produto	Confunde "não agora" com "nunca". É adiamento explícito de escopo.
7ª	128	**F**	Item (II) não precisa de critérios de aceitação (benefício já é claro)	Benefício claro ≠ verificabilidade. É o critério (Gherkin) que torna "pronto" testável (*Testable*).

Versão B — Gabarito: 84 (4 + 16 + 64)

Pos	Val	V/F	Conteúdo (= A)
1ª	2	**F**	[A-7ª] (II) não precisa de critérios
2ª	4	**V**	[A-1ª] (I) viola *Small* / Epic
3ª	8	**F**	[A-5ª] MoSCoW: quase tudo Must
4ª	16	**V**	[A-3ª] Gherkin caminho-triste (II)
5ª	32	**F**	[A-6ª] Won't = descarte definitivo
6ª	64	**V**	[A-4ª] (III) falha em *Valuable*
7ª	128	**F**	[A-2ª] Formato garante valor

Versão C — Gabarito: 74 (2 + 8 + 64)

Alteração: a falsa A-7ª foi substituída por **falsa mais sutil** (caminho triste "opcional" para *Testable*).

Pos	Val	V/F	Conteúdo	Justificativa (só da nova)
1ª	2	**V**	[A-1ª] (I) viola *Small* / Epic	—
2ª	4	**F**	[A-2ª] Formato garante valor	—
3ª	8	**V**	[A-3ª] Gherkin caminho-triste (II)	—
4ª	16	**F**	[A-5ª] MoSCoW: quase tudo Must	—
5ª	32	**F**	**NOVA:** para (II) ser *Testable*, basta o cenário de sucesso; cenários de falha são refinamentos *opcionais* que não afetam a verificabilidade	Trata o caminho triste como opcional. Uma senha "redefinível" que não trata link expirado / senha curta não está de fato verificada. *Testable* exige sucesso **e** falha.
6ª	64	**V**	[A-4ª] (III) falha em *Valuable*	—
7ª	128	**F**	[A-6ª] Won't = descarte definitivo	—

Versão D — Gabarito: 42 (2 + 8 + 32) · 3 corretas

Alteração de cenário: o item **(III)** passa a ser *"Como coordenadora, quero receber alerta quando uma doação estiver próxima da validade, para redistribuí-la"* (benefício real ao usuário). Duas alternativas mudam de V/F ao mesmo tempo (dependem da natureza do item III).

Pos	Val	V/F	Conteúdo	Justificativa das que mudaram
1ª	2	**V**	[A-3ª] Gherkin caminho-triste (II)	—
2ª	4	**F**	[A-6ª] Won't = descarte definitivo	—
3ª	8	**V**	[A-1ª] (I) viola *Small* / Epic	—
4ª	16	**F**	[A-7ª] (II) não precisa de critérios	—
5ª	32	**V**	Item (III) segue o formato **e** entrega valor (evitar perda de doações é	**Mudou F→V:** o novo (III) tem benefício ao usuário; "segue o formato e entrega valor" agora é verdadeiro.

Pos	Val	V/F	Conteúdo	Justificativa das que mudaram
			resultado percebido por quem administra)	
6ª	64	**F**	[A-5ª] MoSCoW: quase tudo Must	—
7ª	128	**F**	Item (III) falha em *Valuable* (perspectiva técnica)	**Mudou V→F:** o novo (III) tem benefício voltado ao usuário; afirmar que falha em *Valuable* por ser "técnico" está incorreto.

QUESTÃO 6 — Qualidade de software: atributos, medição e DoD

(6 alternativas · 3 corretas · 1,0 ponto)

Cenário (A/B): Fim do Sprint 1 do **PetFood**. A busca de produtos passa em todos os critérios de *funcionalidade*, mas leva ~8 segundos com a base real (milhares de produtos). Um integrante diz que o código está "tão embolado" que mudar a ordenação depois será um pesadelo. A equipe vai definir o Definition of Done. *Ruídos:* pair programming; internet do bairro oscila à tarde.

Versão A — Gabarito: 22 (2 + 4 + 16)

Pos	Val	V/F	Alternativa (resumo)	Justificativa
1ª	2	**V**	Rápida nos testes (poucos produtos), mas 8s sob base real → funcionalidade ≠ desempenho	A concessão inicial parece "defender" a busca; a proposição afirma corretamente a distinção entre atributos.
2ª	4	**V**	Código embolado = manutenibilidade; invisível ao usuário mas real p/ a equipe	Embute a tentação ("o usuário nunca vê") e a rejeita. Atributo interno conta.
3ª	8	**F**	Os 8s podem ser a internet do bairro → problema externo, não da equipe	Usa o ruído. A lentidão foi medida com a base real (milhares de produtos) = desempenho sob carga, não a rede.
4ª	16	**V**	DoD com "verificar tempo de resposta sob carga realista" = qualidade contínua	Ressalva ("o DoD anterior não tinha isso") parece brecha, mas reforça: a ausência de critério contínuo causou a descoberta tardia.

Pos	Val	V/F	Alternativa (resumo)	Justificativa
5ª	32	**F**	Passa nos critérios atuais ⇒ "pronta"; desempenho/organização ficam p/ depois	Trata "critérios atuais" (só funcionalidade) como suficiente — o que o DoD existe para evitar.
6ª	64	**F**	Melhor adiar desempenho/manutenibilidade p/ uma verificação final única	Defende a verificação final, que a Semana 4 mostra ser mais cara/menos eficaz que a qualidade contínua.

Versão B — Gabarito: 84 (4 + 16 + 64)

Pos	Val	V/F	Conteúdo (= A)
1ª	2	**F**	[A-6ª] Adiar p/ verificação final
2ª	4	**V**	[A-1ª] Funcionalidade ≠ desempenho
3ª	8	**F**	[A-5ª] Passa nos critérios ⇒ pronta
4ª	16	**V**	[A-2ª] Manutenibilidade (invisível mas real)
5ª	32	**F**	[A-3ª] Culpa da internet
6ª	64	**V**	[A-4ª] DoD com desempenho contínuo

Versão C — Gabarito: 50 (2 + 16 + 32)

Alteração: a falsa A-3ª foi substituída por **falsa mais sutil** (desempenho "subjetivo" por não haver critério).

Pos	Val	V/F	Conteúdo	Justificativa (só da nova)
1ª	2	**V**	[A-1ª] Funcionalidade ≠ desempenho	—
2ª	4	**F**	[A-6ª] Adiar p/ verificação final	—
3ª	8	**F**	**NOVA:** os 8s são lentidão do sistema, mas como nenhum critério fixou tempo máximo, o desempenho é *preferência subjetiva* do cliente, não atributo avaliável objetivamente	Concede o diagnóstico correto (sistema, não rede) para deslizar: desempenho é objetivo (mensurável em tempo de resposta) independentemente de ter sido escrito como critério. A ausência do critério é falha do DoD, não prova de subjetividade.

Pos	Val	V/F	Conteúdo	Justificativa (só da nova)
4ª	16	**V**	[A-2ª] Manutenibilidade	—
5ª	32	**V**	[A-4ª] DoD com desempenho contínuo	—
6ª	64	**F**	[A-5ª] Passa nos critérios ⇒ pronta	—

Versão D — Gabarito: 26 (2 + 8 + 16) · 3 corretas

Alteração de cenário: a busca é **rápida (<0,5s) localmente com a mesma base**, e só fica lenta (8s) **quando acessada pela rede oscilante do pet shop**. A causa isola-se na infraestrutura → duas alternativas mudam de V/F em relação à A.

Pos	Val	V/F	Conteúdo	Justificativa das que mudaram
1ª	2	**V**	[A-4ª] DoD com desempenho contínuo	—
2ª	4	**F**	[A-6ª] Adiar p/ verificação final	—
3ª	8	**V**	Rápida localmente (mesma base), lenta só via rede → a lentidão aponta para infraestrutura externa, não para o sistema	**Mudou F→V** (vs. A-3ª): a medição controlada isola a causa na rede. A atribuição à internet, falsa em A, torna-se correta.
4ª	16	**V**	[A-2ª] Manutenibilidade	—
5ª	32	**F**	[A-5ª] Passa nos critérios ⇒ pronta	—
6ª	64	**F**	O sistema de busca falha em desempenho *independentemente de onde seja executado*	**Mudou V→F** (vs. A-1ª): o sistema é rápido localmente; a lentidão vem da rede. Afirmar falha "independente de onde executa" contraria a evidência de D.

QUESTÃO 7 — Escopo e requisitos assistidos por IA

(7 alternativas · 4 corretas em A/B/C, 5 em D · 1,0 ponto)

Cenário (A/B/C): Aula de escopo assistido por IA. A equipe da ONG **Raízes Urbanas** primeiro **escreveu sozinha, sem IA**, o escopo essencial do caso de treino; depois comparou (versões divergiam) e deu à IA o papel de **analista cético** (apontar personas esquecidas, "problema por trás do problema"). O escopo do MVP do projeto real já fora

aprovado em sprint anterior. *Ruídos:* discussão sobre assistente pago vs. gratuito; ar-condicionado quebrado.

Versão A — Gabarito: 212 (4 + 16 + 64 + 128)

Pos	Val	V/F	Alternativa (resumo)	Justificativa
1ª	2	**F**	Seria mais eficiente abrir a IA **primeiro** e depois divergir a partir da visão dela	Inverte a sequência (humano diverge → IA amplia). Se a IA fala primeiro, a equipe ancora e a divergência genuína (o insight) desaparece.
2ª	4	**V**	Papel de analista cético (perguntar antes de gerar) protege contra o viés de confirmação	Aplica o viés específico e a técnica que o mitiga, sem defini-lo.
3ª	8	**F**	Deixar a IA decidir quais itens entram na fronteira do MVP, com a equipe validando no fim	Fronteira do MVP é passo não-terceirizável. Quem levanta opções (IA) não decide valor (equipe). "Validar no fim" ≠ decidir o recorte.
4ª	16	**V**	Se a equipe não explica cada história sem o assistente, pela política ela ainda não é "sua"	Estende a política de IA ("se não explica ao vivo, não é seu") a histórias, não só código.
5ª	32	**F**	Sobre o escopo aprovado, pedir à IA para *reescrever o Canvas do zero* (versão mais completa)	A regra é **auditar, não recriar**. O escopo aprovado não se refaz por conta da equipe/IA; mudanças se negociam com o stakeholder.
6ª	64	**V**	IA fez boas perguntas ≠ elicitou; lacunas viram **perguntas** à coordenadora, não requisitos já levantados	Reconhece o limite (elicitação real é insubstituível) e o encaminhamento correto do output.
7ª	128	**V**	Manter a divergência individual **antes** da IA preservou o desacordo (o insight que some se a IA apresenta primeiro)	Afirma o valor de divergir-antes (contraponto direto à falsa 1ª).

Versão B — Gabarito: 180 (4 + 16 + 32 + 128)

Pos	Val	V/F	Conteúdo (= A)
1ª	2	**F**	[A-3ª] IA decide a fronteira do MVP
2ª	4	**V**	[A-7ª] Divergir antes preserva o desacordo
3ª	8	**F**	[A-1ª] Abrir a IA primeiro
4ª	16	**V**	[A-2ª] Papel cético / viés de confirmação
5ª	32	**V**	[A-6ª] IA perguntou ≠ elicitou
6ª	64	**F**	[A-5ª] Reescrever o Canvas do zero
7ª	128	**V**	[A-4ª] Explicar ao vivo / histórias

Versão C — Gabarito: 170 (2 + 8 + 32 + 128)

Alteração: a falsa A-3ª foi substituída por **falsa mais sutil** (advogado do diabo → "ceder à IA").

Pos	Val	V/F	Conteúdo	Justificativa (só da nova)
1ª	2	**V**	[A-2ª] Papel cético / viés de confirmação	—
2ª	4	**F**	[A-1ª] Abrir a IA primeiro	—
3ª	8	**V**	[A-4ª] Explicar ao vivo / histórias	—
4ª	16	**F**	**NOVA:** usando a IA como advogado do diabo da fronteira, se a equipe não rebate o contra-argumento, deve *ceder e ajustar a fronteira conforme a IA*	Inverte o papel do advogado do diabo. Não rebater sinaliza justificativa fraca (rever) ou falta de info do stakeholder (vira pergunta), não "obedecer à IA". A decisão é da equipe; a IA testa, não dita.
5ª	32	**V**	[A-6ª] IA perguntou ≠ elicitou	—
6ª	64	**F**	[A-5ª] Reescrever o Canvas do zero	—
7ª	128	**V**	[A-7ª] Divergir antes preserva o desacordo	—

Versão D — Gabarito: 234 (2 + 8 + 32 + 64 + 128) · 5 corretas

Alteração de cenário: o caso passa a ser o **PetFood, caso de treino sem escopo aprovado** (construir Canvas e fronteira do zero). A alternativa sobre "reescrever o Canvas" muda **F→V** (não há artefato aprovado a preservar).

Pos	Val	V/F	Conteúdo	Justificativa da que mudou
1ª	2	**V**	[A-6ª] IA perguntou ≠ elicitou	—
2ª	4	**F**	[A-3ª] IA decide a fronteira do MVP	—
3ª	8	**V**	[A-7ª] Divergir antes preserva o desacordo	—
4ª	16	**F**	[A-1ª] Abrir a IA primeiro	—
5ª	32	**V**	[A-2ª] Papel cético / viés de confirmação	—
6ª	64	**V**	No PetFood (treino, sem escopo aprovado), usar a IA para *estruturar/redigir o Canvas do zero* — com a equipe decidindo o conteúdo — é adequado	**Mudou F→V:** não há artefato aprovado a preservar; construir o Canvas do zero com a IA estruturando (equipe decide) é o proposto na aula. A ressalva "não recriar o aprovado" não se aplica ao treino.
7ª	128	**V**	[A-4ª] Explicar ao vivo / histórias	—

QUESTÃO 8 — Arquitetura em 3 camadas e API como contrato

(5 alternativas · 3 corretas em A/B/C, 2 em D · 1,0 ponto)

Cenário (A/B/C): No **PetFood**, React (front) exibe a lista de produtos; Express expõe `GET /api/produtos`; dados em PostgreSQL. Um integrante colocou no **controller** dessa rota a resposta HTTP, a regra "só retornar produtos com estoque > 0" e o comando SQL — tudo junto. Na Review, a coordenadora pediu que produtos esgotados também apareçam, marcados como indisponíveis. *Ruídos:* idioma da rota (produtos/products); quem sabia SQL.

Versão A — Gabarito: 26 (2 + 8 + 16)

Pos	Val	V/F	Alternativa (resumo)	Justificativa
1ª	2	**V**	O pedido da coordenadora altera a regra de filtragem (no controller); se a regra estivesse num service, a rota e a resposta HTTP ficariam intocadas	Exige simular a mudança: o pedido afeta a regra, não a borda HTTP.
2ª	4	**F**	O frontend teria de esperar o backend mudar o **SQL**, pois é o SQL que o contrato da rota expõe ao cliente	O contrato expõe o formato da *resposta* (JSON), não o SQL interno. Inverte o que a API esconde.
3ª	8	**V**	Com regra + SQL + resposta no mesmo controller, mudar só o JSON obriga a abrir/entender também a regra e a consulta	Consequência concreta da baixa coesão.
4ª	16	**V**	Extrair o SQL para um repositório faz o controller depender da *interface*; trocar o banco afetaria o repositório, não o controller/regra	Inversão de dependência aplicada a uma mudança concreta (trocar banco).
5ª	32	**F**	Como React e Express são processos separados de JS, a regra "estoque > 0" pode ir em qualquer um sem diferença arquitetural	Premissa verdadeira (processos separados), conclusão falsa: regra no front é contornável pelo cliente e não é fonte única de verdade; pertence à lógica no servidor.

Versão B — Gabarito: 52 (4 + 16 + 32)

Pos	Val	V/F	Conteúdo (= A)
1ª	2	**F**	[A-2ª] SQL é o que o contrato expõe
2ª	4	**V**	[A-4ª] Repositório: trocar banco afeta o repo
3ª	8	**F**	[A-5ª] Regra intercambiável front/back
4ª	16	**V**	[A-1ª] Pedido afeta o filtro, não a borda HTTP
5ª	32	**V**	[A-3ª] Mudar só o JSON força ler tudo

Versão C — Gabarito: 50 (2 + 16 + 32)

Alteração: a falsa A-2ª foi substituída por **falsa mais sutil** (mudança interna "obriga" mudar o contrato).

Pos	Val	V/F	Conteúdo	Justificativa (só da nova)
1ª	2	**V**	[A-1ª] Pedido afeta o filtro, não a borda HTTP	—
2ª	4	**F**	**NOVA:** como a rota é um contrato, *qualquer* alteração na regra interna de estoque obriga a alterar também o contrato, pois o cliente "precisa ser avisado"	Mudar a *implementação* por trás do contrato não obriga mudar o *contrato*, desde que o formato da resposta se mantenha compatível. O propósito do contrato é isolar o cliente de mudanças internas.
3ª	8	**F**	[A-5ª] Regra intercambiável front/back	—
4ª	16	**V**	[A-3ª] Mudar só o JSON força ler tudo	—
5ª	32	**V**	[A-4ª] Repositório: trocar banco afeta o repo	—

Versão D — Gabarito: 10 (2 + 8) · 2 corretas

Alteração de cenário: o código **já foi refatorado** em camadas (controller só recebe/responde; regra num *service*; SQL num *repository*). A alternativa que pressupõe o SQL ainda no controller muda **V→F**.

Pos	Val	V/F	Conteúdo	Justificativa da que mudou
1ª	2	**V**	[A-3ª, ajustada] Antes da refatoração, mudar só o JSON obrigava a abrir regra e SQL juntos — problema que a separação atual elimina	Permanece V; ajuste apenas contextual.
2ª	4	**F**	[A-5ª] Regra intercambiável front/back	—
3ª	8	**V**	[A-1ª, ajustada] Com a regra no service, o pedido da coordenadora se concentra no service, deixando controller e rota intocados	Permanece V, mais nítido no cenário refatorado.

Pos	Val	V/F	Conteúdo	Justificativa da que mudou
4ª	16	**F**	[A-2ª] SQL é o que o contrato expõe	—
5ª	32	**F**	O SQL **ainda** está no controller; seria preciso extraí-lo para um repositório como próximo passo	**Mudou V→F:** no cenário D o SQL **já foi extraído** para um repository. Afirmar que "ainda está no controller" contradiz o enunciado.

QUESTÃO 9 — Princípios de design: coesão, acoplamento, SoC e nomes

(6 alternativas · 4 corretas em A/B/C, 3 em D · 1,0 ponto)

Cenário (A/B/C): Na equipe da ONG **Raízes Urbanas**, dois integrantes trabalham em `voluntarioController.js` (~250 linhas), que contém validação de CPF, envio de e-mail, acesso ao banco e resposta HTTP — tudo numa função `handle()`. No merge, o GitHub acusou vários conflitos no mesmo trecho. Um deles precisou mudar só o texto do e-mail e teve de reler a função inteira. *Ruídos:* horários de trabalho alternados; temas de editor diferentes.

Versão A — Gabarito: 106 (2 + 8 + 32 + 64)

Pos	Val	V/F	Alternativa (resumo)	Justificativa
1ª	2	**V**	Com resposta + validação + SQL no mesmo corpo, duas correções na mesma semana recaem em linhas vizinhas → conflito de merge	Afirma o mecanismo do conflito sem nomear "SoC".
2ª	4	**F**	Mesmo turno faria os conflitos *deixarem de ocorrer* (sempre partiriam da versão mais recente)	Integração frequente reduz divergência, mas não elimina conflito quando ambos editam o mesmo trecho entrelaçado. Falsa de ponta a ponta.
3ª	8	**V**	Extrair a validação de CPF permite testar CPF malformado sem disparar e-mail nem conectar ao banco	Consequência factual da extração (teste isolado).
4ª	16	**F**	Renomear `handle()` → `handleCadastroVoluntario`	Primeira parte razoável (nome melhor), conclusão falsa: a causa é a

Pos	Val	V/F	Alternativa (resumo)	Justificativa
			`()` seria *suficiente* para resolver a manutenção	mistura de responsabilidades, não o nome.
5ª	32	**V**	Mover o e-mail para `enviarEmailBoasVindas()` permite trocar o provedor mexendo só nela, sem reabrir validação/resposta	Consequência do desacoplamento aplicada a mudança concreta.
6ª	64	**V**	Dividir 250→125 linhas mantendo a mistura reduz o tamanho, mas dois devs ainda colidem, agora espalhados por dois arquivos	Volume ≠ responsabilidade; dividir por linha não resolve a colisão.

Versão B — Gabarito: 108 (4 + 8 + 32 + 64)

Pos	Val	V/F	Conteúdo (= A)
1ª	2	**F**	[A-4ª] Renomear resolve a manutenção
2ª	4	**V**	[A-1ª] Conflito nasce do corpo compartilhado
3ª	8	**V**	[A-6ª] Dividir por tamanho espalha a colisão
4ª	16	**F**	[A-2ª] Mesmo turno elimina os conflitos
5ª	32	**V**	[A-3ª] Extrair CPF → testar isolado
6ª	64	**V**	[A-5ª] E-mail em função → trocar provedor sem reabrir o resto

Versão C — Gabarito: 114 (2 + 16 + 32 + 64)

Alteração: a falsa A-4ª foi substituída por **falsa mais sutil** (extrair só o CPF "basta").

Pos	Val	V/F	Conteúdo	Justificativa (só da nova)
1ª	2	**V**	[A-1ª] Conflito nasce do corpo compartilhado	—
2ª	4	**F**	[A-2ª] Mesmo turno elimina os conflitos	—
3ª	8	**F**	**NOVA:** extrair *só* a validação de CPF já bastaria para resolver a manutenção, pois as demais	Extrair 1 das 4 responsabilidades não faz as outras 3 pararem de se misturar — continuam no mesmo

Pos	Val	V/F	Conteúdo	Justificativa (só da nova)
			partes deixariam de interferir umas nas outras	corpo, ainda colidindo. "Isolar uma parte" ≠ "separar todas".
4ª	16	**V**	[A-6ª] Dividir por tamanho espalha a colisão	—
5ª	32	**V**	[A-3ª] Extrair CPF → testar isolado	—
6ª	64	**V**	[A-5ª] E-mail em função → trocar provedor sem reabrir o resto	—

Nota de coerência (C-Q9): a nova falsa 3ª ("extrair só o CPF resolve") convive com a verdadeira 5ª ("extrair o CPF permite testá-lo isolado"). Não se contradizem: extrair o CPF traz o ganho de testabilidade local (5ª, V) **e** não resolve a mistura das outras três (3ª, F por dizer que resolve).

Versão D — Gabarito: 42 (2 + 8 + 32) · 3 corretas

Alteração de cenário: o arquivo **já foi desmembrado** — `validarCpf.js`, `enviarEmailBoasVindas.js`, `voluntarioRepository.js`, e o controller só orquestra. A alternativa sobre "conflito enquanto tudo compartilha o corpo" muda **V→F**.

Pos	Val	V/F	Conteúdo	Justificativa da que mudou
1ª	2	**V**	[A-3ª] Extrair CPF → testar isolado (agora concretizado)	Permanece V.
2ª	4	**F**	[A-2ª] Mesmo turno elimina os conflitos	—
3ª	8	**V**	[A-5ª] E-mail isolado → trocar provedor mexendo só nessa parte	Permanece V, mais concreto.
4ª	16	**F**	[A-4ª] Renomear resolve a manutenção	—
5ª	32	**V**	[A-6ª] Dividir por tamanho espalharia a colisão (princípio geral)	Permanece V.
6ª	64	**F**	Enquanto as 4 responsabilidades compartilharem o corpo	**Mudou V→F:** após a refatoração, as responsabilidades não compartilham mais o mesmo corpo.

Pos	Val	V/F	Conteúdo	Justificativa da que mudou
			da mesma função, dois devs colidirão — "situação que ainda descreve o estado atual"	Quem muda o e-mail mexe só em `enviarEmailBoasVindas.js`.

QUESTÃO 10 — Estilos arquiteturais, trade-offs e C4/ADR

(7 alternativas · 4 corretas em A/B/C, 3 em D · 1,0 ponto)

Cenário (A/B/C): Equipe de 4 pessoas, 4 meses, sistema de reservas de biblioteca universitária, hoje monólito Client/Server (React + Express em camadas + 1 banco). Na Review, levaram um **diagrama de contexto** (Aluno e Bibliotecário usando o Sistema de Reservas, que se conecta ao Sistema de E-mail da universidade) e registraram o **ADR-003 "Banco único"** (Contexto: equipe de 4, 4 meses, sem experiência distribuída; Decisão: um PostgreSQL; Consequências: simplifica agora, revisar se crescer muito). *Ruídos:* vídeo de "big tech"; pergunta sobre app de celular (fora de escopo).

> **Natureza das alternativas (Q10 e Q11):** cada alternativa é uma **cadeia argumentativa** com elos individualmente plausíveis. O julgamento exige avaliar a articulação/o domínio de validade do princípio, não casar uma definição.

Versão A — Gabarito: 102 (2 + 4 + 32 + 64)

Pos	Val	V/F	Alternativa (resumo)	Justificativa
1ª	2	**V**	Separar reservas/catálogo agora por causa do Bounded Context seria erro: o BC governa *como* delimitar serviços que a equipe decidiu ter, não é razão para introduzir serviços num contexto (4p/4m) que não os pede	O princípio pressupõe já se ter optado por serviços distribuídos; não decide *se* dividir.
2ª	4	**V**	Apagar o campo *Consequências* do ADR-003 enfraqueceria o registro; é ele que impede a equipe futura de tratar a escolha como definitiva	Aplica a Primeira Lei editando o artefato: a consequência-custo dá valor durável ao ADR.
3ª	8	**F**	Só rotular tecnologias sobre as caixas do diagrama de Contexto já o converte em diagrama de Contêiner	O que distingue o N2 não é rotular tecnologia sobre atores/sistemas externos, mas representar as unidades

Pos	Val	V/F	Alternativa (resumo)	Justificativa
				executáveis *internas* e sua comunicação — que o N1 não contém.
4ª	16	**F**	Como o bibliotecário não opinou no técnico (e perguntou do app), o diagrama de Contexto foi inadequado; melhor ter levado o de Contêiner	O N1 é o nível *certo* para um não técnico validar contexto/fronteiras; mais detalhe (N2) o afastaria. A pergunta do app é ruído de escopo.
5ª	32	**V**	Antecipar a separação de serviços impõe custo certo agora (equipe sem experiência, 4 meses) para evitar um custo futuro apenas possível e condicional ao crescimento	Pesa custo presente/certo × futuro/incerto.
6ª	64	**V**	Registrar o ADR-003 não é desperdício mesmo que mude: um novo ADR registrará a mudança e o ADR-003 seguirá explicando o contexto original (histórico, evita rediscutir do zero)	ADRs não são apagados quando superados; acumulam-se como histórico.
7ª	128	**F**	Declarar o monólito "sem trade-offs" prova objetivamente que é a decisão certa	Pela Primeira Lei, não enxergar desvantagem indica custo ainda não percebido. Endossa o *raciocínio* errado — falsa mesmo sendo o monólito, de fato, adequado.

Versão B — Gabarito: 212 (4 + 16 + 64 + 128)

Pos	Val	V/F	Conteúdo (= A)
1ª	2	**F**	[A-7ª] "Sem trade-off prova acerto"
2ª	4	**V**	[A-1ª] BC governa *como*, não *se* dividir
3ª	8	**F**	[A-4ª] Bibliotecário não opinou ⇒ levar N2
4ª	16	**V**	[A-6ª] Novo ADR supersede; ADR-003 vira histórico
5ª	32	**F**	[A-3ª] N2 = rotular tecnologia sobre o N1

Pos	Val	V/F	Conteúdo (= A)
6ª	64	**V**	[A-2ª] Apagar *Consequências* enfraquece o ADR
7ª	128	**V**	[A-5ª] Custo certo agora × possível/condicional

Versão C — Gabarito: 202 (2 + 8 + 64 + 128)

Alteração: a falsa A-4ª foi substituída por **falsa mais sutil** (aprovação do N1 "chancela" corretude técnica).

Pos	Val	V/F	Conteúdo	Justificativa (só da nova)
1ª	2	**V**	[A-1ª] BC governa *como*, não *se* dividir	—
2ª	4	**F**	[A-7ª] "Sem trade-off prova acerto"	—
3ª	8	**V**	[A-2ª] Apagar *Consequências* enfraquece o ADR	—
4ª	16	**F**	**NOVA:** como o N1 é o nível certo para o não técnico, a aprovação do bibliotecário valida que a **arquitetura interna** está tecnicamente bem construída	O N1 não expõe decisão interna alguma. A aprovação dele diz respeito a *resolver o problema certo para as pessoas certas*, não a *estar bem construído por dentro*. Confunde validação de escopo com validação técnica.
5ª	32	**F**	[A-3ª] N2 = rotular tecnologia sobre o N1	—
6ª	64	**V**	[A-6ª] Novo ADR supersede; ADR-003 vira histórico	—
7ª	128	**V**	[A-5ª] Custo certo agora × possível/condicional	—

Versão D — Gabarito: 38 (2 + 4 + 32) · 3 corretas

Alteração de cenário: o projeto passa a ser **equipe de 20 pessoas / vários times / 3 anos de roadmap / crescimento de 10× já contratado**. A cadeia "custo certo × custo possível" (A-5ª) muda **V→F** (o custo futuro deixou de ser condicional).

Pos	Val	V/F	Conteúdo	Justificativa da que mudou
1ª	2	**V**	[A-6ª] Novo ADR supersede; ADR-003 vira histórico	—
2ª	4	**V**	[A-2ª] Apagar *Consequências* enfraquece o ADR	—
3ª	8	**F**	[A-3ª] N2 = rotular tecnologia sobre o N1	—
4ª	16	**F**	[A-7ª] "Sem trade-off prova acerto"	—
5ª	32	**V**	[A-1ª] BC governa *como*, não *se* dividir	—
6ª	64	**F**	[A-4ª] Bibliotecário não opinou ⇒ levar N2	—
7ª	128	**F**	Recusar antecipar a separação sob o argumento "o custo futuro é só possível e condicional"	**Mudou V→F:** com 20 pessoas, múltiplos times e crescimento de 10× *já contratado*, o custo futuro é praticamente certo e iminente, e há capacidade operacional. O trade-off agora pende para antecipar; recusar sob "custo só possível" está incorreto.

> **Destaque pedagógico (Q10-D):** a *mesma* cadeia é **V no contexto pequeno (A) e F no grande (D)** — a resposta certa depende do contexto (Primeira Lei). Quem decorou "antecipar microsserviços é sempre exagero" erra na D.

QUESTÃO 11 — Git: colaboração, branches, merge e PRs (BÔNUS)

(5 alternativas · 1 correta · valor: BÔNUS, soma por cima de 10,0)

Cenário (A/B/C): Equipe da ONG **Raízes Urbanas**, 4 pessoas no mesmo repositório; cada feature em um branch, integrada à `main` via PR revisado. Na semana: **A** manteve um branch 6 dias sem integrar (muitas mudanças); **B** abriu um PR pequeno mas ninguém revisou por 3 dias; **C/D** trabalharam no mesmo arquivo em branches diferentes e adiaram a integração para o último dia. Houve conflitos grandes e demorados. *Ruídos:* cadência de commit dos integrantes; `main` protegida contra push direto.

> Cada alternativa é uma cadeia argumentativa; as falsas quebram por função-vs-forma, causa-vs-lugar, regra-vs-represamento ou princípio-usado-ao-contrário.

Versão A — Gabarito: 4 (só a 2ª)

Pos	Val	V/F	Alternativa (resumo)	Justificativa
1ª	2	**F**	PR de B travou 3 dias → autorizar o autor a aprovar/integrar o próprio PR para o fluxo não travar	Autoaprovar preserva o rito do PR e elimina o que o justifica (outro olhar). A resposta ao gargalo é destravar a *revisão*, não remover o revisor.
2ª	4	**V**	O branch longo de A e a integração de última hora de C/D têm a **mesma causa**: deixaram a distância até a `main` crescer; o esforço de merge foi acumulado nos dias anteriores, não criado no dia do conflito	Unifica dois sintomas sob a causa real (integração tardia) e localiza o custo corretamente. **Única correta.**
3ª	8	**F**	Como o conflito foi no mesmo arquivo, organizar as tarefas para que dois nunca toquem o mesmo arquivo na Sprint eliminaria o conflito na origem	"Mesmo arquivo" é onde o conflito *apareceu*, não por que foi grande/tardio. Engessa a divisão de tarefas e falha quando dois toques legítimos recaírem no mesmo módulo.
4ª	16	**F**	A proteção da `main` (exige branch+PR) foi o que represou as mudanças; afrouxá-la para push direto em urgências reduziria conflitos	A proteção obriga a *passar por PR*, não a *demorar dias*. O represamento veio de adiar a integração, não da regra. Afrouxar instabiliza a `main` sem tocar a causa.
5ª	32	**F**	Padronizar todos para commitar 1×/dia alinharia a cadência e, com uma janela diária única, concentraria e reduziria os conflitos	Usa o princípio "integração frequente → conflitos menores" para chegar ao oposto (reduzir a frequência). O certo é integrar *mais* vezes.

Versão B — Gabarito: 16 (só a 4ª)

Pos	Val	V/F	Conteúdo (= A)
1ª	2	**F**	[A-5ª] Todos 1 commit/dia
2ª	4	**F**	[A-3ª] Nunca dois no mesmo arquivo
3ª	8	**F**	[A-1ª] Autoaprovar o próprio PR
4ª	16	**V**	[A-2ª] A e C/D: mesma causa (integração tardia)

Pos	Val	V/F	Conteúdo (= A)
5ª	32	**F**	[A-4ª] Afrouxar a proteção da `main`

Versão C — Gabarito: 2 (só a 1ª)

Alteração: a falsa A-3ª foi substituída por **falsa mais sutil** (coordenação no chat "substitui" integração frequente).

Pos	Val	V/F	Conteúdo	Justificativa (só da nova)
1ª	2	**V**	[A-2ª] A e C/D: mesma causa (integração tardia)	**Única correta.**
2ª	4	**F**	[A-1ª] Autoaprovar o próprio PR	—
3ª	8	**F**	[A-5ª] Todos 1 commit/dia	—
4ª	16	**F**	**NOVA:** bastaria a dupla avisar no chat quem edita qual arquivo antes de começar; com isso, poderiam seguir em branches separados por vários dias sem conflito grande	Avisar quem edita o quê evita *colisão simultânea no mesmo trecho*, mas não impede os branches de divergirem da `main` por dias. O conflito grande vem da divergência acumulada, que só a integração frequente reduz. Comunicação ≠ integração contínua.
5ª	32	**F**	[A-4ª] Afrouxar a proteção da `main`	—

Versão D — Gabarito: 8 (só a 3ª) · experimento controlado

Alteração de cenário: C/D passam a integrar várias vezes ao dia (conflitos pequenos, resolvidos na hora); só **A** adiou. O merge de A é grande; o de C/D, tranquilo. A cadeia que unia A e C/D (correta em A) vira **falsa** — e entra uma nova correta que isola a variável frequência.

Pos	Val	V/F	Conteúdo	Justificativa
1ª	2	**F**	[A-1ª] Autoaprovar o próprio PR	—
2ª	4	**F**	A e C/D compartilham a causa de integração tardia	**Falsa em D:** C/D integraram com frequência; não acumularam divergência. Unificar com A é

Pos	Val	V/F	Conteúdo	Justificativa
				falso aqui. (Distrator forte para quem decorou o gabarito de A.)
3ª	8	**V**	O contraste entre o merge demorado de A e o tranquilo de C/D — mesmo repositório, mesmas regras — indica que o fator decisivo do custo do merge foi a **frequência de reconciliação** com a `main`, não usar branch nem tocar arquivos compartilhados	**Correta (nova):** lê o experimento natural (mesmas condições, resultados opostos) e isola a variável certa.
4ª	16	**F**	[A-4ª] Afrouxar a proteção da `main`	—
5ª	32	**F**	[A-3ª] Nunca dois no mesmo arquivo	—

Conferência final de gabaritos

Q	A	B	C	D
Q1	12	18	36	42
Q2	38	84	82	10
Q3	90	108	114	22
Q4	42	52	50	58
Q5	26	84	74	42
Q6	22	84	50	26
Q7	212	180	170	234
Q8	26	52	50	10
Q9	106	108	114	42
Q10	102	212	202	38
Q11 (bônus)	4	16	2	8

Todos os 4 gabaritos de cada questão são distintos entre si.